

웨딩올인원

AI 웨딩플랜 · OpenClaw · 쇼핑/커뮤니티 · 업체 탐색/예약 · 준비관리 · Restful API

팀장 황용현
팀원 이재원 · 권용익 · 윤승진

Team GitHub:

https://github.com/sai0734/Sixfour_Team

Team YouTube:

https://youtu.be/ltr_3dvikuc?si=hor0HbVLppz1mMK-

목차

Project Overview

- 프로젝트 개요
- 기획 배경 및 문제 정의
- 서비스 목표 및 기대 효과

Architecture

- 사용자 역할 및 서비스 흐름
- 시스템 아키텍처 / 기술 스택
- ERD / 팀원 역할 분담

Features

- 업체탐색 / 비교 / 예약 / 결제
- 준비관리 / 답례품 / 쇼핑몰 / 커뮤니티
- 반응형 웹 구현 및 AWS 배포
- 트러블 슈팅 및 회고

프로젝트 기획 배경

문제 인식

웨딩홀 대관료·식대·스드메 가격 구조가 제각각이고,
정확한 견적을 받으려면 여러 업체를 직접 상담해야 합니다.

- 가격이 계약 후에야 공개되는 정보 비대칭
- 지역·업체별 비용 격차로 합리적 비교가 어려움
- 홀·스드메·예산·일정이 사이트마다 흩어져 있음
- 반복 탐색 비용이 만혼·비혼 부담으로 이어짐

→ 가격 투명성 + 준비 과정 통합이 필요합니다.

참고 근거

- ① 조선일보(2023) 서울 웨딩홀 90곳 중 약 8할 인상
스드메 300만원대 → 500만원 안팎 상승 언급
- ② 세계일보(2019) 평균 결혼비용 약 2억3186만원
예식장 가격 비공개·추가비로 비교 자체가 어려움

참고 주소 URL:

<https://www.segye.com/newsView/20190410517854>

https://www.chosun.com/national/national_general/2023/02/14/JYSI-CKCUXBC5LOZOB2YGZQLEQ/

핵심 수치

식대 평균
약 6.5만원

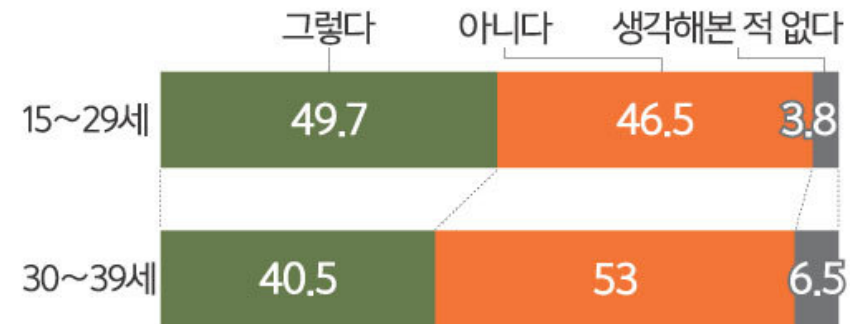
전국 대관료 평균
약 389만원

강남 대관료 평균
약 700만원

참고 사진

경제적 부담에 결혼을 망설인 적이 있나 (단위: %)

*15~39세 남녀 2500명 대상 설문조사



자료: 한국청소년정책연구원

문제 정의 및 해결 방향

문제 1

정보 분산

웨딩홀·스튜디오·드레스·메이크업 정보가 여러 사이트/블로그/카페에 흩어짐



해결 1

원스톱 흐름

검색 → 비교 → 찜 → 예약 → 결제까지 한 플랫폼에서 완결

문제 2

수기 관리 한계

예산·일정을 엑셀/메모앱으로 관리 → 누락·중복 발생



해결 2

준비관리 통합

체크리스트 + 예산 + D-day 연동으로 진행상황을 한눈에

문제 3

비교 번거로움

동일 조건으로 비교가 어려움



하이라이트

AI 웨딩플랜 자동 추천

예산·날짜·하객수·스타일 입력만으로 AI 웨딩플랜(업체 조합) 자동 추천

문제 4

문의 채널 파편화

전화·카카오톡·인스타DM 등으로 분산



해결 4

소통 통합

실시간 문의 채팅으로 업체-회원 커뮤니케이션을 플랫폼 내로 통합

기술 스택

Frontend

React 18.3 · Vite / Redux Toolkit · React Router
Axios · Tailwind CSS / STOMP/SockJS · Recharts

Backend

Spring Boot 3.5 · Java 21 / Spring Security · Spring Data JPA
JWT · WebSocket · Spring Mail / Validation · ModelMapper

Data & AI

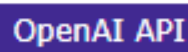
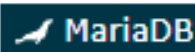
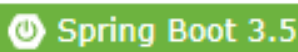
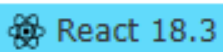
MariaDB · Redis / OpenAI API · GPT-4o-mini
Kakao Login · Toss Payments / Daum 우편번호 API

AWS

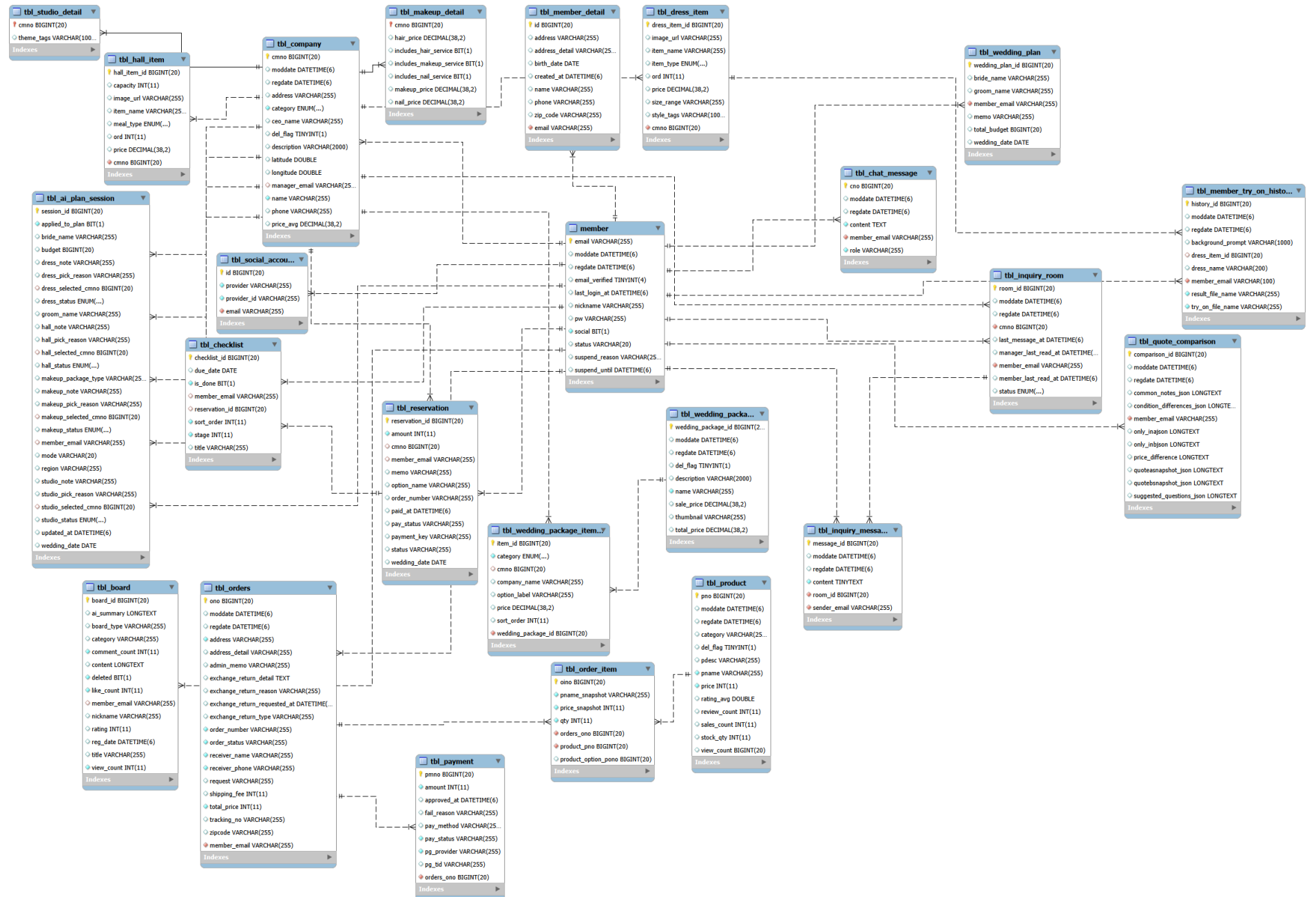
EC2 · RDS · IAM · S3 · Elastic Beanstalk
Nginx · Linux · GitHub Actions

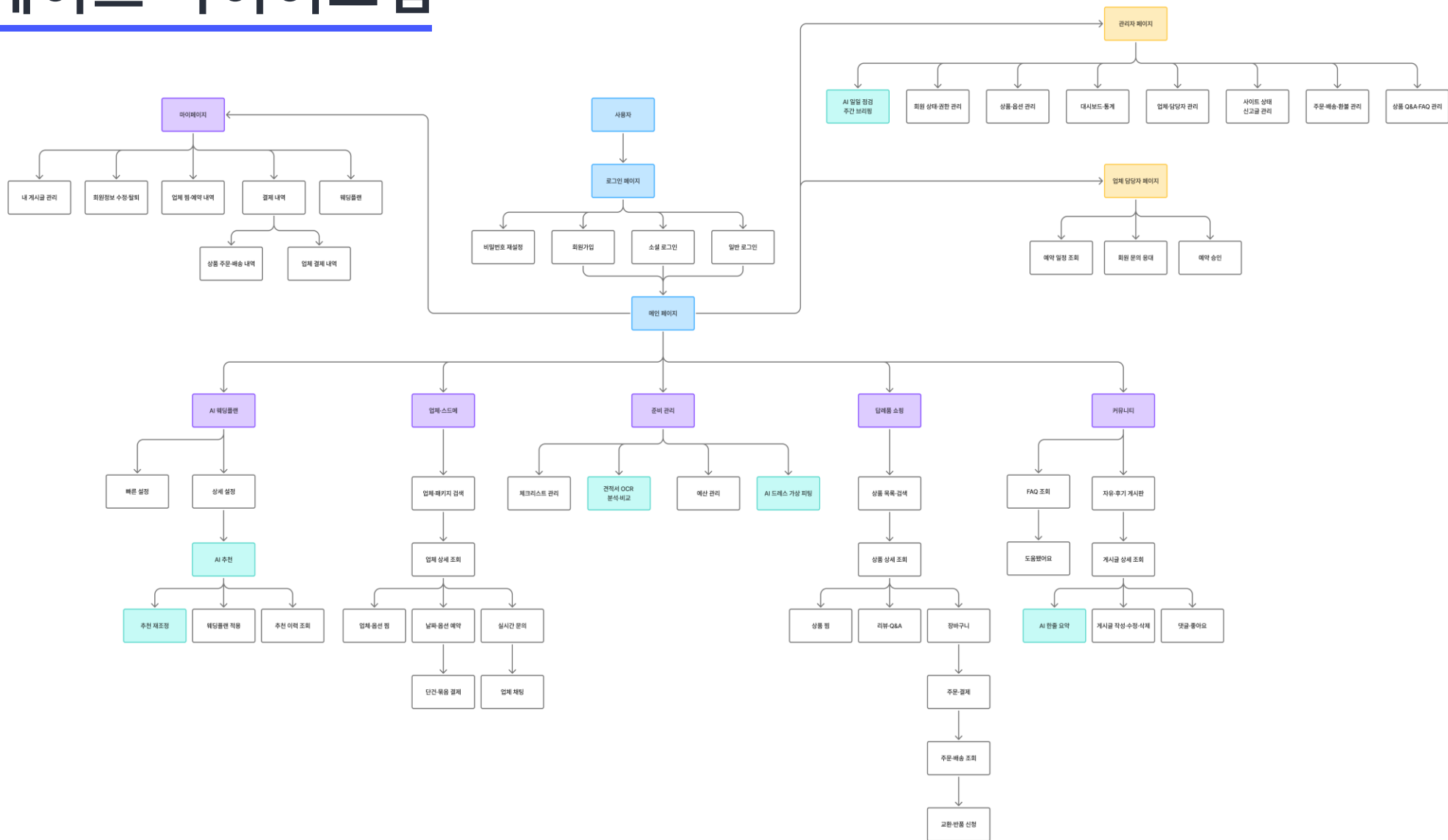
이름	담당 및 주요 구현 기능
황용현 팀장	•답례품 쇼핑몰(장바구니/주문/결제) — REST CRUD + Toss Payments 연동 •업체문의 실시간 채팅 — WebSocket(STOMP) + SockJS •AI챗봇 — OpenAI Function/Tool Calling •OpenClaw — AI 대시보드 주간브리핑/게시글,리뷰 욕설탐지/PDF생성
권용익	•로그인 — JWT(Access/Refresh) + Spring Security + Redis(토큰/블랙리스트 TTL) •회원가입 — Spring Security(BCrypt) + 이메일 인증(EmailVerify JSON 임시저장) •회원관리·업체담당자 관리자 페이지, 관리자 대시보드(통계/차트) •커뮤니티 AI 한줄요약 기능 — 프롬프트 설계, 캐싱 로직 구현
이재원	•AI 웨딩플랜 — OpenAI Chat Completions + 서버 슬롯 상태머신 •커뮤니티 게시판 — OpenAI 한줄요약(캐싱) •준비관리(체크리스트/예산/웨딩플랜), 예약, 찜(업체), FAQ, AI 견적서(quote) — Google Vision OCR + OpenAI
윤승진	•업체관리(홀/드레스/스튜디오/메이크업 CRUD) — JPA + 파일 업로드 •AI 드레스 가상피팅 — 외부 CatVTON 이미지합성 API + OpenAI gpt-image-2(배경 교체) •업체 예약 일부

Core Stack



핵심 ERD

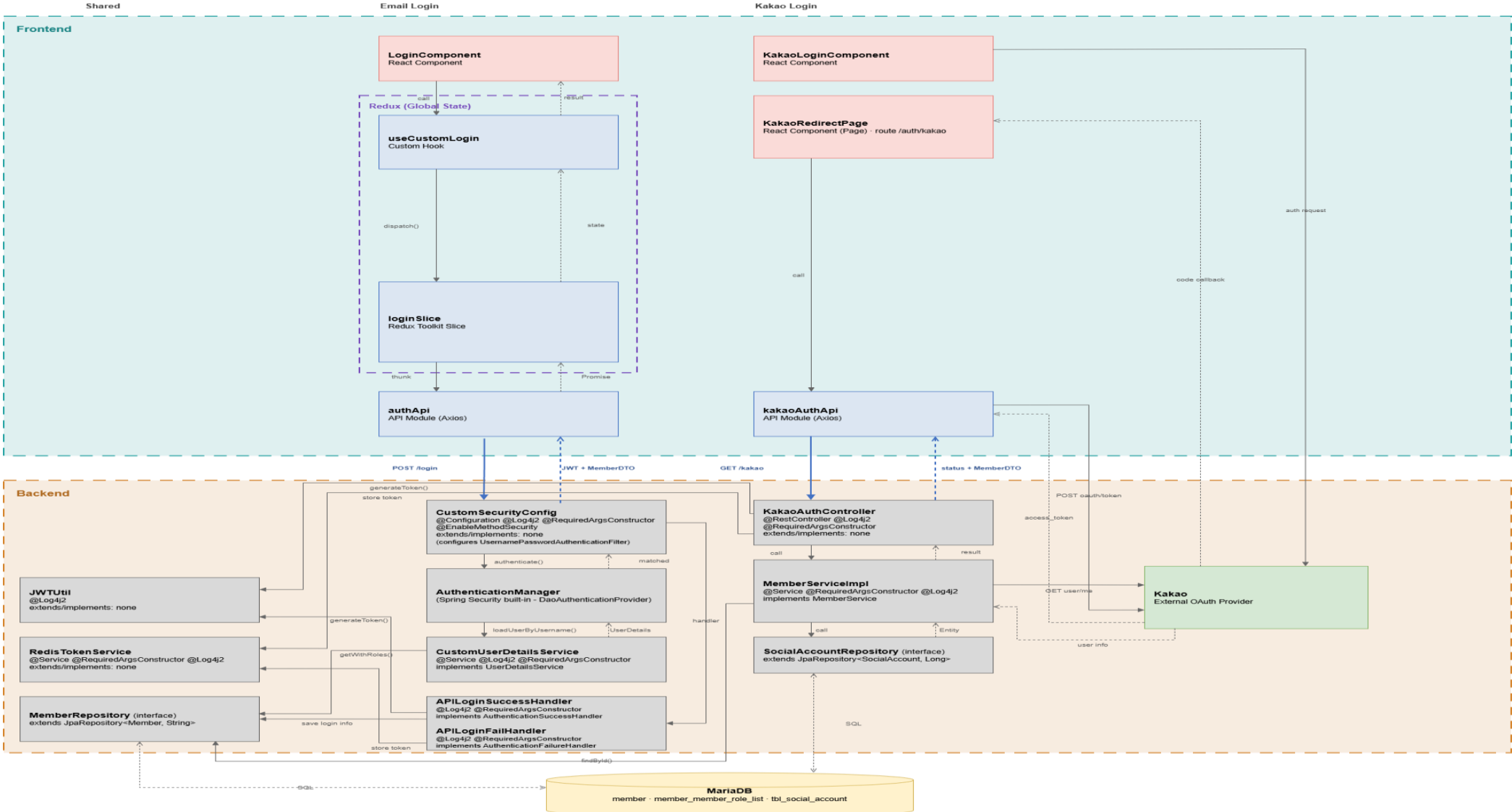




WBS

구분	No.	Task	시작일	종료일	6/30~7/5						7/6~7/12						7/13~7/19						7/20~7/26						7/27~7/29			
					6/30 화	7/1 수	7/2 목	7/3 금	7/4 토	7/5 일	7/6 월	7/7 화	7/8 수	7/9 목	7/10 금	7/11 토	7/12 일	7/13 월	7/14 화	7/15 수	7/16 목	7/17 금	7/18 토	7/19 일	7/20 월	7/21 화	7/22 수	7/23 목	7/24 금	7/25 토	7/26 일	7/27 월
A	1	프로젝트 초기 세팅 (Git 전략, DB 연결)	2026-06-30	2026-06-30																												
A	2	ERD 설계 및 데이터베이스 규칙 통일	2026-06-29	2026-07-01																												
B	1	회원가입 · 이메일 인증	2026-07-01	2026-07-02																												
B	2	로그인 · JWT/Redis · 로그인 실패 잠금	2026-07-02	2026-07-08																												
B	3	카카오 소셜 로그인 재설계	2026-07-08	2026-07-10																												
B	4	회원관리 · 관리자 페이지	2026-07-06	2026-07-09																												
B	5	업체 담당자 권한 · 계정설정 통합	2026-07-13	2026-07-14																												
C	1	업체(홈/드레스/스튜디오/메이크업) CRUD · 데이터	2026-06-30	2026-07-09																												
C	2	업체 상세페이지 · 카카오톡 연동	2026-07-10	2026-07-13																												
C	3	준비관리 (체크리스트 · 예산 · D-day)	2026-07-01	2026-07-10																												
C	4	예약 · 환불제 연동	2026-07-13	2026-07-16																												
D	1	답례품 상품 상세 · 목록 · 필터	2026-07-01	2026-07-06																												
D	2	장바구니 · 결제(Toss Payments)	2026-07-04	2026-07-08																												
D	3	관리자 상품 · 주문 관리	2026-07-06	2026-07-10																												
D	4	상품 교환 · 반품	2026-07-13	2026-07-14																												
E	1	커뮤니티 게시판 CRUD	2026-07-02	2026-07-09																												
E	2	커뮤니티 AI 한줄요약	2026-07-16	2026-07-16																												
E	3	업체문의 채팅 (REST 폴링 → WebSocket)	2026-07-13	2026-07-16																												
F	1	AI 웨딩플랜 (추천 · 세션 · 리파인)	2026-07-16	2026-07-22																												
F	2	AI 챗봇 (Function Calling)	2026-07-07	2026-07-20																												
F	3	AI 드레스 가상피팅	2026-07-16	2026-07-23																												
F	4	AI 견적서 (OCR 비교)	2026-07-23	2026-07-23																												
G	1	중간 점검 및 버그 목록화	2026-07-17	2026-07-17																												
G	2	전체 플로우 통합 테스트	2026-07-20	2026-07-21																												
H	1	OpenClaw 운영관제(일일점검/주간브리핑)	2026-07-22	2026-07-24																												
H	2	AWS 배포 (EC2 · Elastic Beanstalk · RDS)	2026-07-24	2026-07-28																												
I	1	발표 PPT · PDF 작성	2026-07-21	2026-07-24																												
I	2	README · ERD · 트러블슈팅 문서화	2026-07-24	2026-07-28																												
I	3	발표 리허설	2026-07-24	2026-07-27																												
I	4	최종 발표	2026-07-29	2026-07-29																												

JWT + Spring Security + Redis



로그인 및 소셜로그인

WEDDING ALL IN ONE

소중한 날을 위한 특별한 시작

로그인하고 나만의 맞춤 웨딩 플랜과 일정을 확인해보세요.

로그인

웨딩을 위한 서비스에 오신 것을 환영합니다.

이메일

example@email.com

이메일은 필수 입력 항목입니다.

비밀번호

비밀번호를 입력해주세요

☐ 로그인 유지 [비밀번호 찾기](#)

로그인하기

간편 로그인

카카오로 로그인

계정이 없으신가요? [회원가입하기](#)

```
http.addFilterBefore(new JWTCheckFilter  
    (redisTokenService, memberRepository, memberDetailRepository),  
    UsernamePasswordAuthenticationFilter.class); //JWT 체크
```

로그인 성공/실패를 커스텀 핸들러로 가로채고, JWT 검증 필터를 앞단에 끼워 넣어 세션 없는 REST 인증 흐름을 구성하였습니다.

```
if ("READY".equals(result.status()))  
String jwtAccessToken =  
    JWTUtil.generateToken(claims, JWTUtil.ACCESS_TOKEN_MINUTES);
```

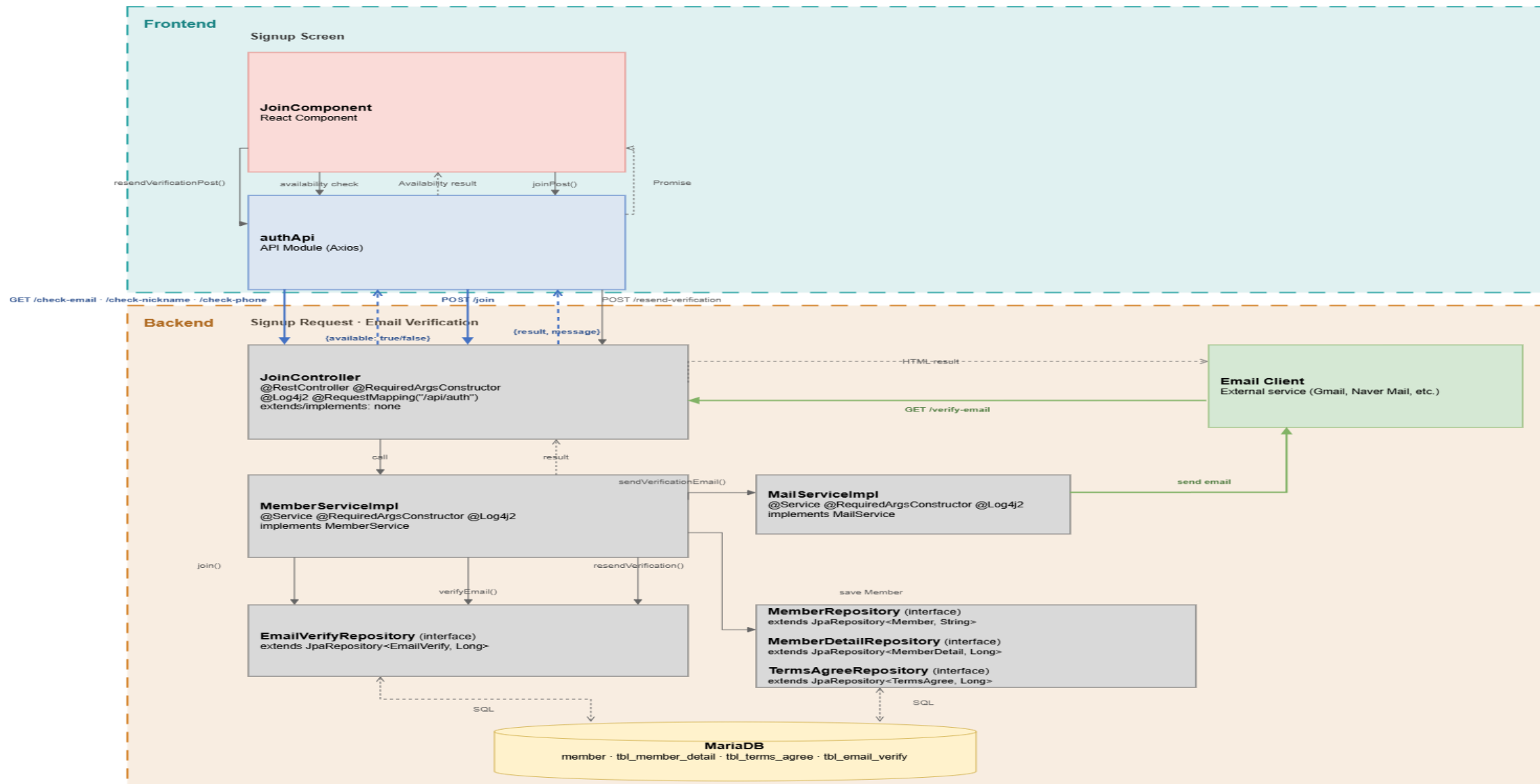
카카오 인증만으로는 로그인 시키지 않고, 가입 완료 시점에만 진짜 로그인(JWT 발급) 처리합니다.

```
String accessToken = JWTUtil.generateToken  
    (claims, JWTUtil.ACCESS_TOKEN_MINUTES);  
redisTokenService.saveRefreshToken  
    (memberDTO.getEmail(), refreshToken, rememberMe);
```

Access/Refresh 만료시간을 상수로 표준화하고, refresh 토큰은 Redis에 TTL과 함께 저장해 자동 정리되도록 구성하였습니다.

회원가입 Flow

Spring Security + JWT



회원가입

회원가입

환영합니다,
새 계정을
만들어보세요

가입 후 모든 서비스를 자유롭게 이용하세요

계정 만들기

정보를 입력해 가입을 완료하세요

이메일 *

hong@example.com

비밀번호 *

비밀번호 확인 *

닉네임 *

이름 *

휴대폰 번호 *

생년월일

주소 *

우편번호

주소 검색

주소 검색을 눌러주세요

상세주소를 입력해 주세요

☐ 전체 동의

☐ [필수] 이용약관에 동의합니다.

☐ [필수] 개인정보처리방침에 동의합니다.

☐ [선택] 마케팅 정보 수신에 동의합니다.

가입하기

이미 계정이 있으신가요? 로그인

```
if (memberRepository.findById(emailVerify.getEmail()).isPresent())  
    String phoneStatus = getPhoneCheckStatus(joinDTO.getPhone());
```

가입 신청과 이메일 인증 사이에 시간차가 있기 때문에, 그 사이에 같은 이메일/휴대폰으로 다른 경로 가입이 먼저 끝났을 가능성을 인증 확정 직전에 한 번 더 검사합니다.

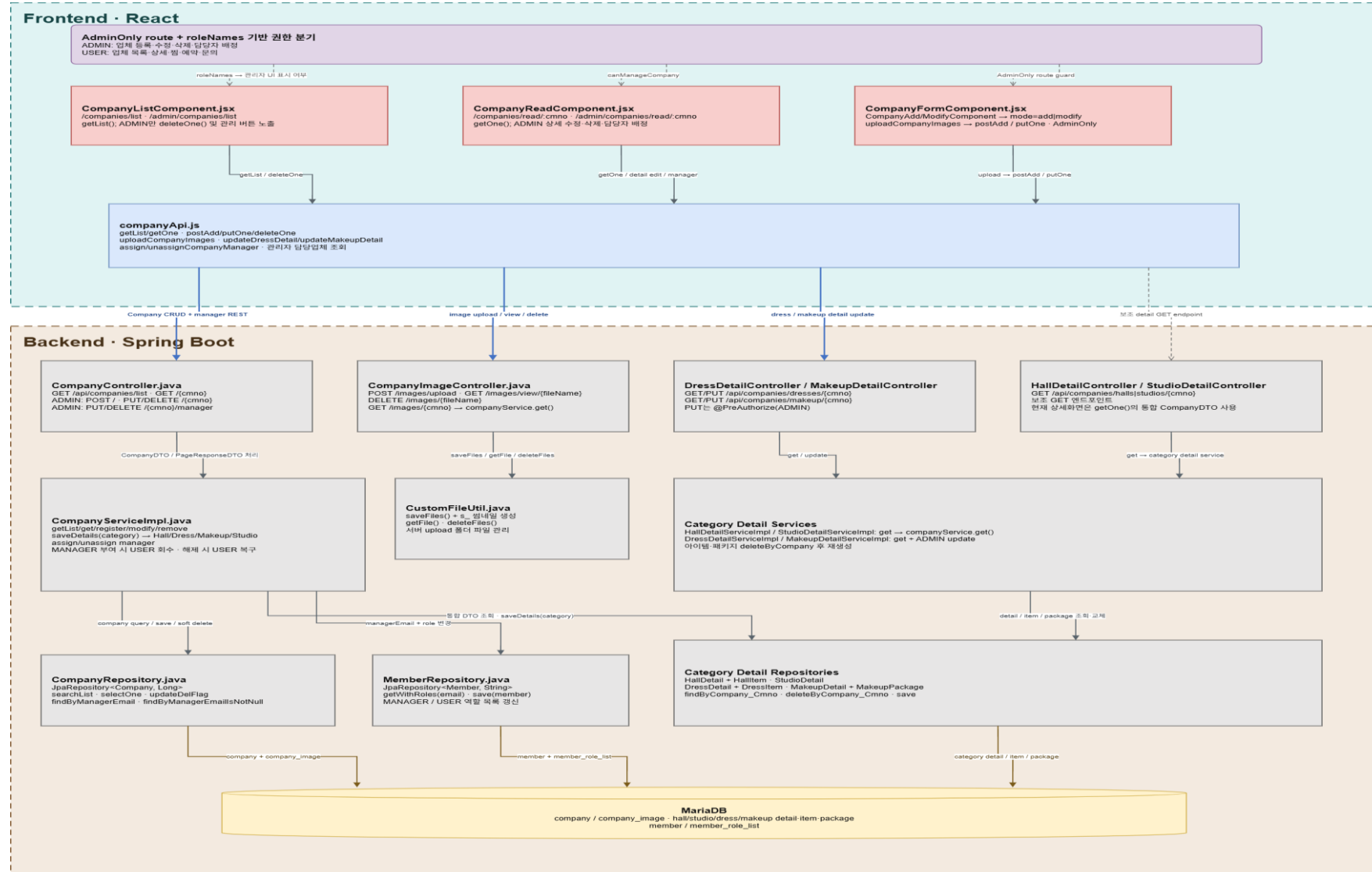
```
joinDTO.setPw(passwordEncoder.encode(joinDTO.getPw()));  
EmailVerify emailVerify = EmailVerify.builder()  
    .payload(payload)
```

회원가입 요청 시점엔 Member 테이블에 아무것도 안 만들고, 입력값 전체를 암호화된 비밀번호와 함께 JSON으로 EmailVerify에 임시 저장합니다.

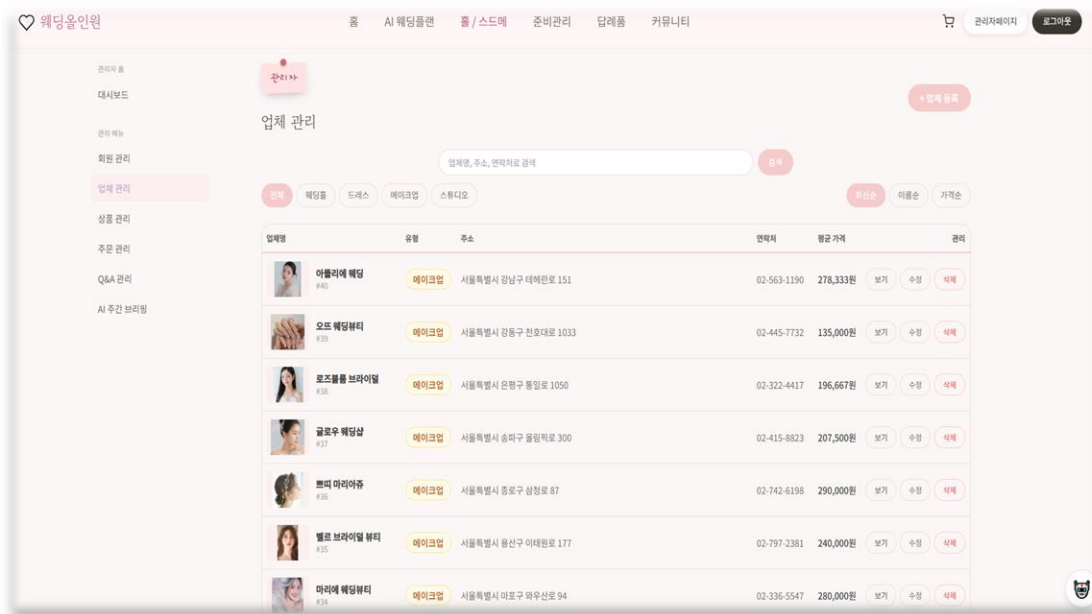
```
Optional<EmailVerify> result =  
    emailVerifyRepository.findTopByEmailOrderByIdDesc(email);  
EmailVerify previous = result.get();  
EmailVerify emailVerify = EmailVerify.builder()  
    .email(email)  
    .payload(previous.getPayload())  
    .expiredAt(LocalDate.now().plusMinutes(30))  
    .build();
```

인증 메일을 못 받았거나 30분이 지나 만료됐을 때, 회원가입 폼을 처음부터 다시 채우게 하지 않고 이전에 저장해둔 payload(입력값 JSON)를 그대로 재사용하면서 토큰과 만료시간만 새로 발급합니다

업체관리 Flow Spring Data JPA



업체관리



```
@GetMapping("/list") @User DESKTOP-OT238HQWEZEN
public PageResponseDTO<CompanyListDTO> list(CompanySearchDTO searchDTO) {
    log.info(searchDTO);
    return companyService.getList(searchDTO);
}
```

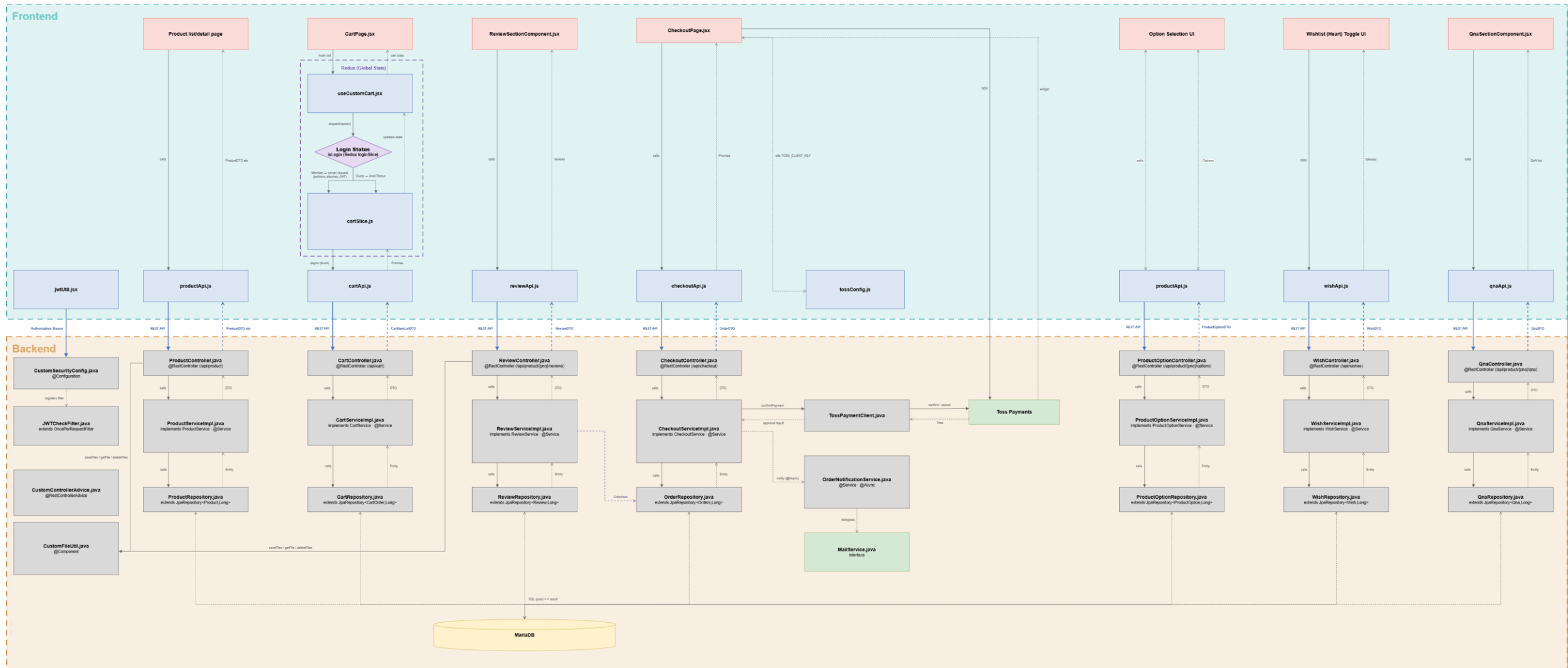
목록/검색/유형/필터, 등록/보기/수정/삭제가 컨트롤러의 list/CRUD 엔드포인트와 연결됩니다.

```
const isAdmin = loginState.roleNames?.some((r) =>
    ["ADMIN", "ROLE_ADMIN"].includes(r),
const companyListPath =
    isAdmin ? "/admin/companies/list" : "/companies/list";
@PreAuthorize("hasRole('ADMIN')")
```

roleNames에 ADMIN이 있는지로 관리자 여부를 판단해서, 같은 업체 목록이라도 이동 경로를 /admin/companies/list(관리자용)와 /companies/list(일반유저용)로 나누고, 등록/수정/삭제 API는 @PreAuthorize("hasRole('ADMIN')")로 서버 단에서 한 번 더 관리자 권한을 검증합니다.

답례품 Flow

REST CRUD + Toss Payments



답례품

CART

장바구니

전체선택 (1/1)

선택삭제

도자기 호작도 금술잔 2

컬러 4p

색상: 정색

- 1 +

45,000원 ×

결제 예정 금액

상품금액 45,000원

배송비 무료

총액 45,000원

결제하기 (1)

CHECKOUT

주문/결제

주문 상품

도자기 호작도 금술잔 2컬러 4p (정색) × 1

45,000원

배송지 정보

최근 배송지 불러오기

항상전

010-1234-5678

06035

우편번호 찾기

서울 강남구 가로수길 5

4층 402호

신속 정박

상품금액 45,000원

배송비 무료

결제 금액 45,000원

결제하기

결제 방법을 선택해주세요

간편결제

pay

카카오페이

SSG PAY

SSG페이

더 보기 >

신용·체크카드

신한

하나Pay

우리

Kbank

케이뱅크

토스뱅크

카카오뱅크

KB국민(KB pay)

비씨(마이빅)

농협(NH페이)

더 보기 >

상품명

도자기 호작도 금술잔 2컬러 4p

결제 금액

45,000원

실제 결제가 되는 테스트용 카드

toss payments

```
export const mergeGuestCartAsync = createAsyncThunk(
  "mergeGuestCartAsync",
  async (email, { getState }) => {
    const state = getState();
    const guestItems = state.cartSlice.filter((item) => item.isGuest);

    for (const item of guestItems) {await postChangeCart(
      {email,pno: item.pno,pono: item.pono,qty: item.qty,cino: null,});}

    const serverItems = await getCartItems();
    return serverItems;
  },
);
```

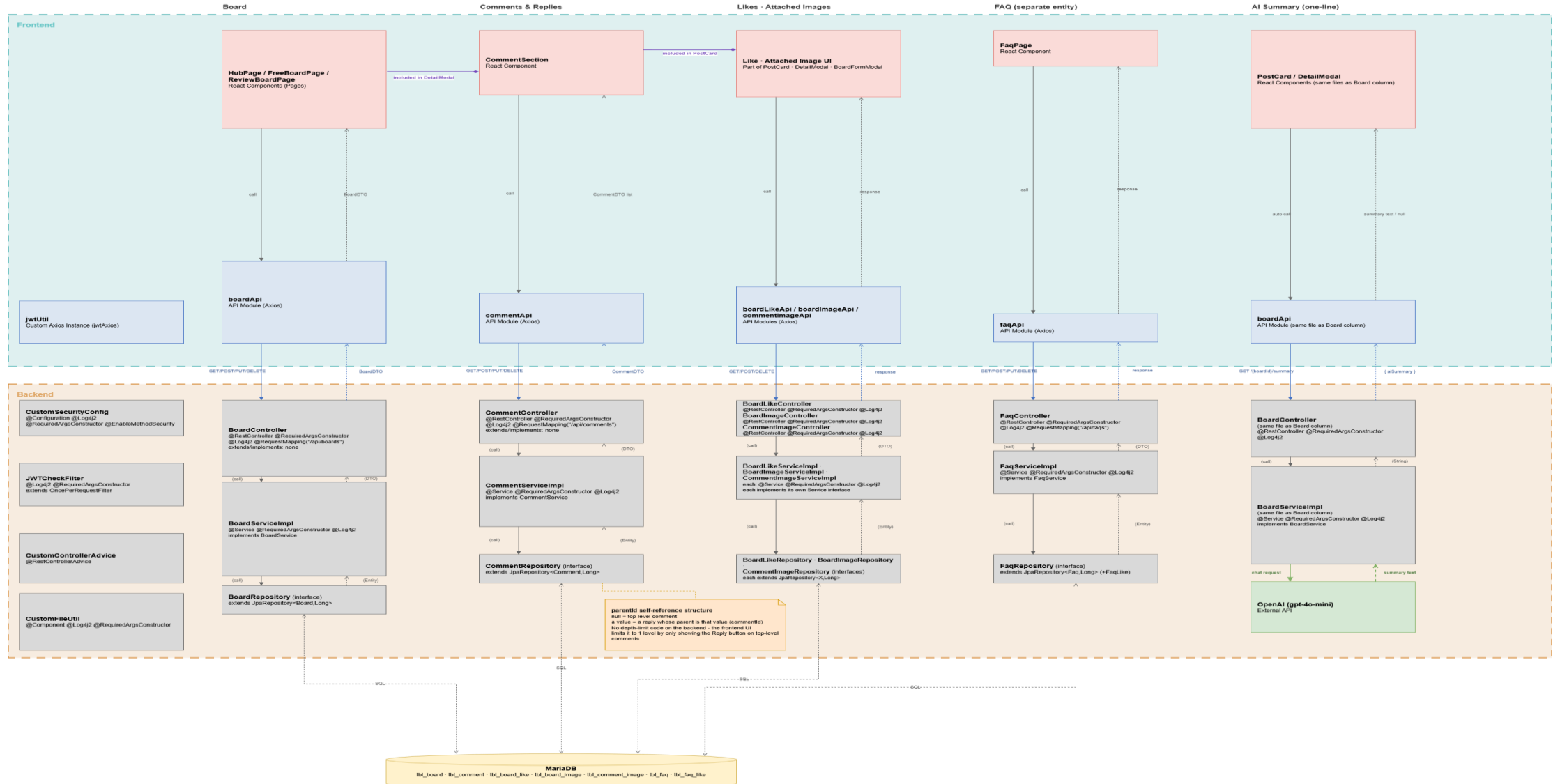
로그인 전엔 장바구니를 Redux 상태로만 유지하다가 로그인하는 순간 서버 DB에 옮깁니다. 이때 여러 상품을 한꺼번에 보내지 않고 for...of와 await를 사용해 하나씩 순서대로 처리해서 꼬이는걸 방지했습니다.

```
orders.changeTotalPrice(totalPrice);
if (orders.getTotalPrice() !== confirmRequestDTO.getAmount()) {
  throw new IllegalStateException
    ("결제 금액이 주문 금액과 일치하지 않습니다.");
}
```

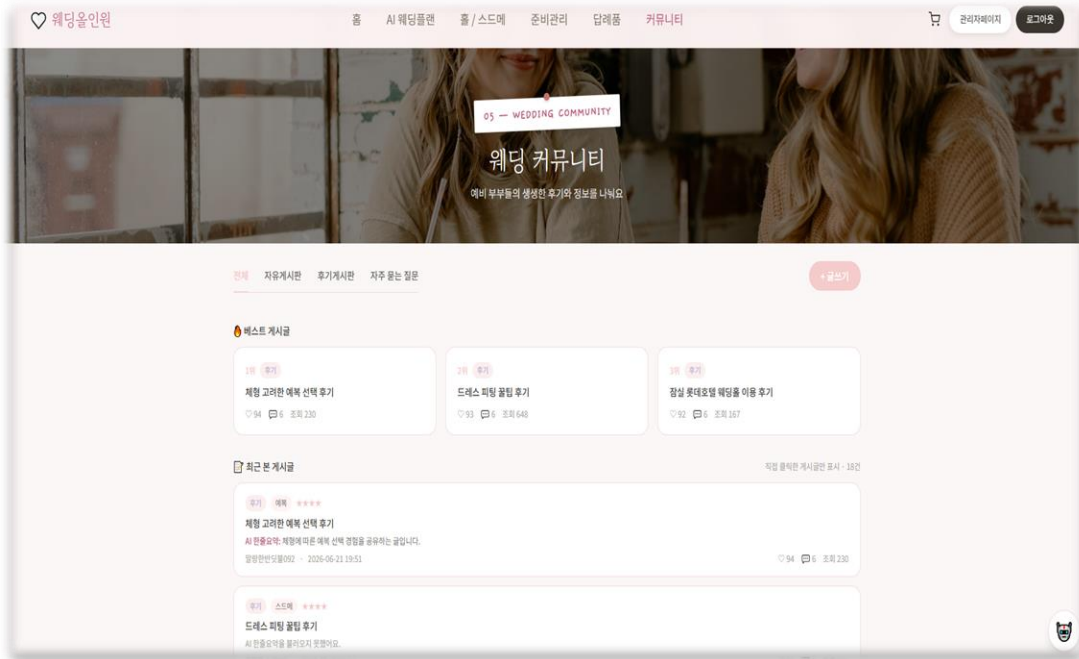
결제창을 띄우기 전 서버에서 먼저 주문을 만들어서 실제 금액을 저장하고 결제 승인 시점에 저장된 금액과 클라이언트가 보낸 금액을 대조합니다. 이 절차가 없으면 클라이언트 쪽 요청을 조작하여 실제 금액과 다른 금액이 결제될 수도 있습니다.

커뮤니티 및 AI 한줄요약 Flow

Spring Data JPA + OpenAI Chat Completions API



커뮤니티 및 AI 한줄요약



```
const topLevel = comments.filter((c) => !c.parentId); if (!comment.isDeleted()) {
const repliesOf = (parentId) =>
  comments.filter((c) => c.parentId === parentId);
  comment.changeDeleted(true);
  commentRepository.save(comment);
```

서버는 댓글을 시간순 평평한 목록으로만 내려주고, parentId 유무로 최상위 댓글/대댓글을 나눠 프론트에서 트리 구조로 렌더링 - 부모 댓글이 삭제돼도 소프트 삭제 처리라 대댓글은 그대로 살아있습니다.

```
if (board.getAiSummary() != null && !board.getAiSummary().isBlank()) {
  return board.getAiSummary(); // 이미 있으면 OpenAI 호출 없이 바로 반환
}

OpenAiResponseDTO response = openAiClient.getChatCompletions(messages, tools: null);
String summary =
  response.getChoices().getFirst().getMessage().getContent().trim();

board.changeAiSummary(summary); // DB에 캐싱
boardRepository.save(board);
```

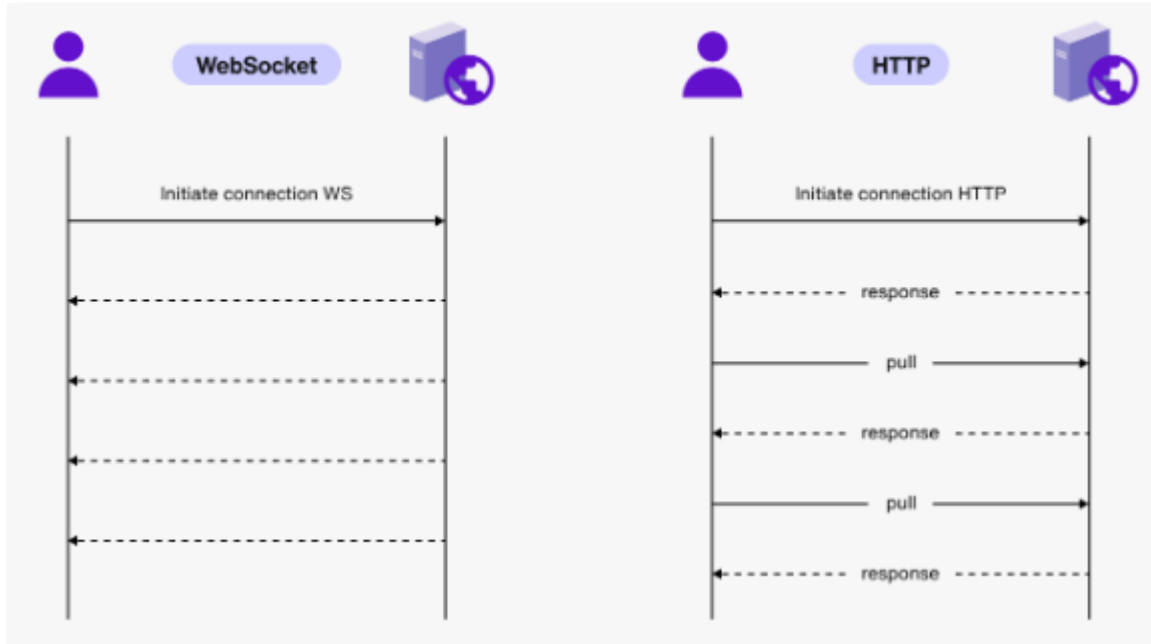
게시글마다 딱 한 번만 OpenAI를 호출하고 결과를 Board.AISummary 컬럼에 저장 -> 이후 조회는 DB 캐시로 응답, 비용.응답속도 문제를 함께 해결. 글 수정 시엔 changeAiSummary(null)로 캐시를 비워 다음 조회 때 재 생성되게 처리합니다.

```
useEffect(() => {
  if (!isSummaryEligible || post.aiSummary || post.boardId == null) return;
  // 캐시 있거나 짧으면 호출 안 함
  getAiSummary(post.boardId).then(...);
}, [post.boardId, post.aiSummary, isSummaryEligible]);
```

본문 150자 미만인 글은 요약이 별 의미가 없다고 판단해 API 요청 자체를 생략, 목록 응답에 이미 캐시된 요약이 있으면 그것도 재요청하지 않음 -> 불필요한 OpenAI호출을 프론트 단에서부터 줄입니다.

업체 문의 Flow

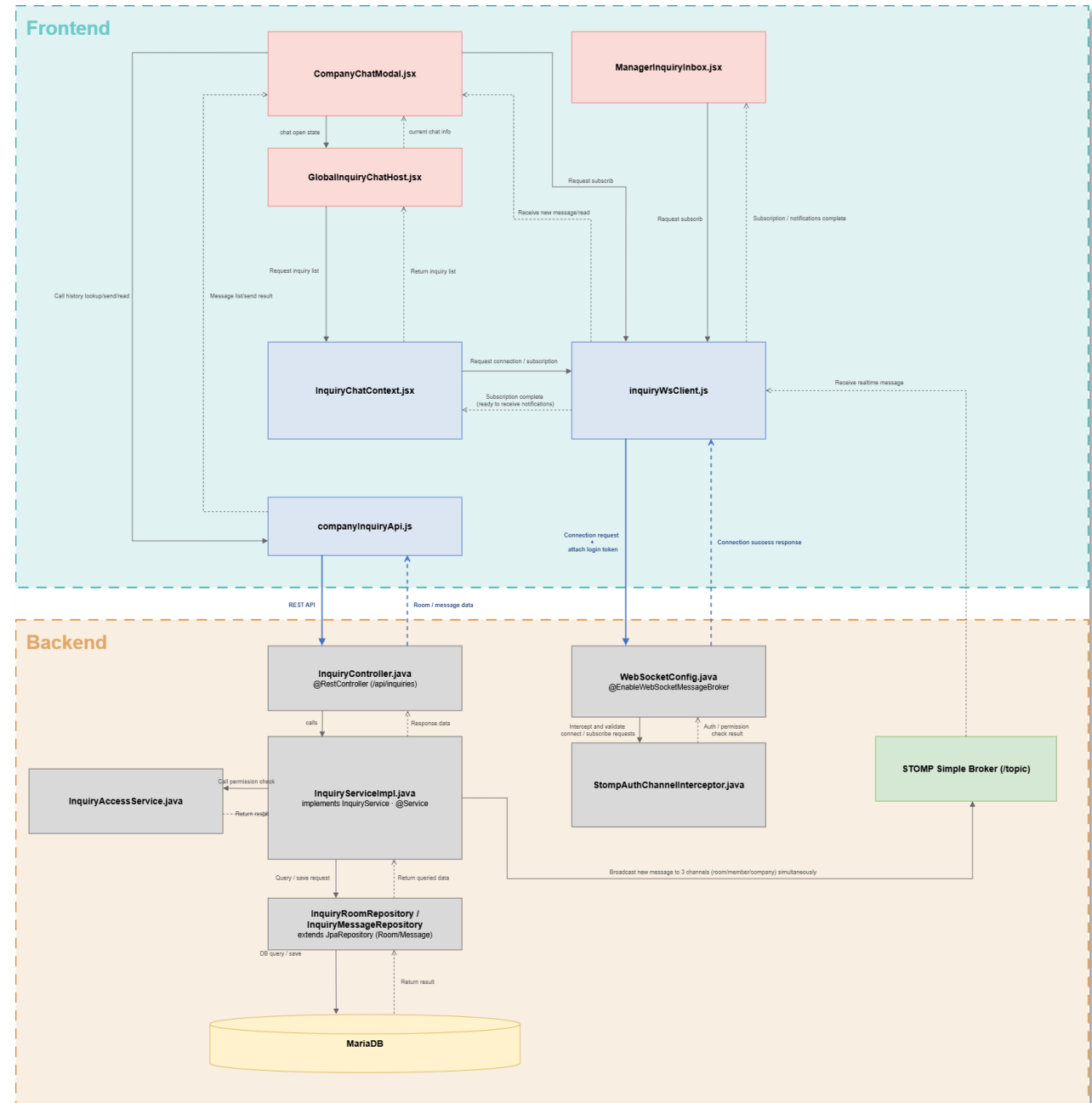
WebSocket(STOMP) + SockJS
JWT-dver+STOMP인증



WebSocket이란?

웹 브라우저와 서버 간의 전이중(full-duplex)통신채널을 제공하는 컴퓨터 통신 프로토콜입니다. HTTP와 달리 한 번 연결이 수립되면 클라이언트와 서버 양방향으로 데이터를 동시에 그리고 실시간으로 주고 받을 수 있습니다.

- 실시간 통신 : 연결을 유지하며 지연 시간을 최소화
- 양방향 통신 : 서버도 클라이언트에게 데이터를 먼저 보낼 수 있다.(서버푸시)
- 효율적인 통신 : 헤더 오버헤드를 줄여 데이터 전송 효율을 높입니다.



업체 문의



```
InquiryMessage saved = inquiryMessageRepository.save(message);
room.updateLastMessageAt(now);

messagingTemplate.convertAndSend(
    destination: "/topic/inquiries/room/" + roomId, InquiryRoomEventDTO.message(messageDTO));

InquiryRoomDTO memberView = InquiryRoomDTO.from(room, room.getMemberEmail());
messagingTemplate.convertAndSend( destination: "/topic/inquiries/member/" + room.getMemberEmail(), memberView);
InquiryRoomDTO managerView = InquiryRoomDTO.from(
    room, MANAGER_VIEWER_MARKER, resolveMemberNickname(room.getMemberEmail()));
messagingTemplate.convertAndSend( destination: "/topic/inquiries/company/" + room.getCmno(), managerView);
```

메시지 저장은 REST API로 처리하며 저장이 끝난 후 STOMP를 이용해 새 메시지의 신호를 3곳(채팅방 화면, 회원알리벳지, 업체 매니저함)에 동시에 뿌려줍니다.
즉 저장은 REST API, 알림만 STOMP로 나눠놓은 구조입니다.

```
} else if (destination.startsWith("/topic/inquiries/member/")) {
    String targetEmail = destination.substring("/topic/inquiries/member/".length());
    if (!callerEmail.equals(targetEmail)) {
        throw new AccessDeniedException("본인 알림만 구독할 수 있습니다.");
    }
}
```

구독(SUBSCRIBE)요청이 올 때 마다 해당 요청인이 방 / 업체 / 이메일 알림을 볼 권한이 있는지를 검증하는 로직입니다.

```
client = new Client({
    websocketFactory: () => new SockJS(`${API_SERVER_HOST}/ws-inquiry`),
    reconnectDelay: 5000,
    beforeConnect: () => {client.connectHeaders = buildConnectHeaders();},
    onConnect: () => {
        liveSubscriptions.clear();
        listenersByDestination.forEach((_callbacks, destination) => {resubscribe(destination)});
    }
});
```

부득이한 상황으로 인해 인터넷이 끊겼을 경우 5초마다 재연결을 시도하여 자동으로 알림을 다시 보여줍니다.

최적화 ① - N+1 방지 — 목록과 이미지를 한 번에 조회

최적화 코드 · @EntityGraph / FetchType.LAZY

```
@EntityGraph(attributePaths = "imageList") 7개 사용 위치 DESKTOP-OT238HQWEZEN
@Query("""
select c from Company c
where c.delFlag = false
and (:category is null or c.category = :category)
and (:keyword is null or c.name like concat('%', :keyword, '%') or c.address like concat('%', :keyword, '%'))
and (:minPrice is null or c.priceAvg >= :minPrice)
and (:maxPrice is null or c.priceAvg <= :maxPrice)
""")
```

```
@MapsId
@OneToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "cmno")
private Company company;

private String hallName;
private String address;
private Double latitude;
private Double longitude;
private String phone;
private String representative;
```

적용 화면 · 홀/스드메 업체 목록

업체명	유형	주소	연락처	평균 가격	관리
아틀리에 웨딩 #40	메이크업	서울특별시 강남구 테헤란로 151	02-563-1190	278,333원	보기
오프 웨딩뷰티 #39	메이크업	서울특별시 강동구 천호대로 1033	02-445-7732	135,000원	보기
로즈블룸 브라이덜 #38	메이크업	서울특별시 은평구 통일로 1050	02-322-4417	196,667원	보기
글로우 웨딩샵 #37	메이크업	서울특별시 송파구 올림픽로 300	02-415-8823	207,500원	보기
뽀띠 머리아쥬 #36	메이크업	서울특별시 종로구 삼청로 87	02-742-6198	290,000원	보기

핵심

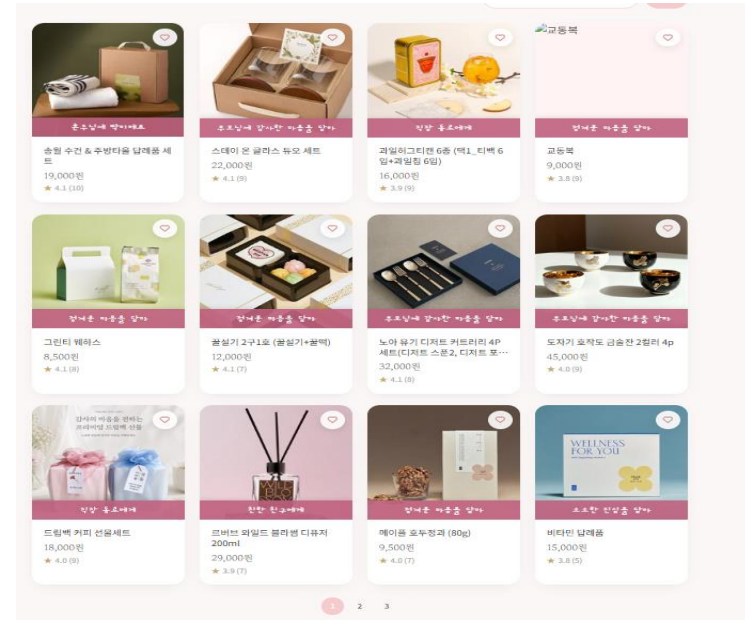
Companyrepository의 조회 목록 메서드에 @EntityGraph(attributePaths = imageList)를 적용, 업체목록을 가져올때 이미지도 같이 가져온다.
가져온 첫번째 이미지를 메인으로 선택하여 CompanyListDTO로 변환, 프론트에서 업체목록에서 이미지와 함께 표시를 한다.
홀과 스튜디오의 상세정보를 FetchType.LAZY로 유지하면서 목록에서 불필요한 데이터까지의 조회를 막아놓아서 효율적으로 전달하도록 한다.

최적화 ② - 서버 사이드 페이징 — Page 단위로만 조회

최적화 코드 · Pageable / PRODUCT_LIST_PAGE_SIZE

```
Page<Object[]> result = productRepository.searchProductList(  
    categories,  
    productSearchDTO.getKeyword(),  
    productSearchDTO.getMinPrice(),  
    productSearchDTO.getMaxPrice(),  
    productSearchDTO.getMinRating(),  
    pageable);  
  
long totalCount = result.getTotalElements();  
  
return PageResponseDTO.<~>withAll()  
    .dtoList(dtoList)  
    .pageRequestDTO(productSearchDTO)  
    .totalCount(totalCount)  
    .build();
```

적용 화면 · 답례품 상품 목록



핵심

- 상품을 검색할 때 조건(카테고리·가격·평점·검색어)에 맞는 상품 전체를 전부 가져오는게 아니라, 현재 화면에 보여줄 한 페이지 분량만 DB에서 가져온다.
- 조회 결과에서 전체 개수(totalCount)만 따로 뽑아 PageResponseDTO에 함께 담아 응답 → 프론트는 실제 목록 데이터를 다시 요청하지 않고도 이 값 하나로 페이지 번호 네비게이션을 그릴 수 있다.

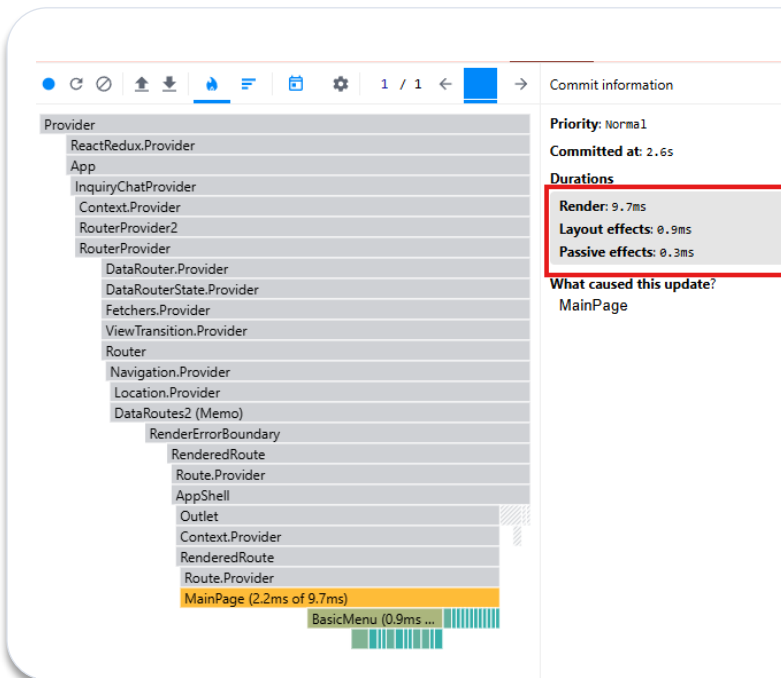
최적화 ③ - 라우트 단위 Code Splitting (React.lazy)

`lazy(() => import(...)) + <suspense fallback={...}>`

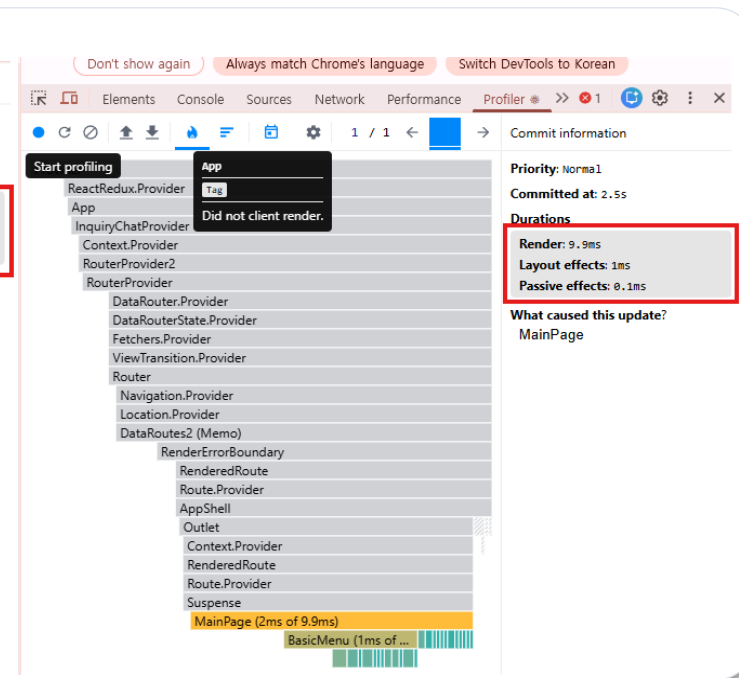
```
const AdminDashboard = lazy(() => import("../pages/admin/AdminDashboardPage"));
const AdminMemberManage = lazy(() => import("../pages/admin/MemberManagePage"));
const ChecklistListPage = lazy(() => import("../pages/checklist/ListPage"));
const PrepHubPage = lazy(() => import("../pages/prep/HubPage"));
const PrepAiDressPage = lazy(() => import("../pages/prep/AiDressPage"));
const PrepQuotePage = lazy(() => import("../pages/prep/QuotePage"));
```

```
const appRoutes = [
  {
    path: "/",
    element: (
      <Suspense fallback={Loading}>
        <Main />
      </Suspense>
    ),
  },
  {
    path: "/about",
    element: (
      <Suspense fallback={Loading}>
        <About />
      </Suspense>
    ),
  },
];
```

Before



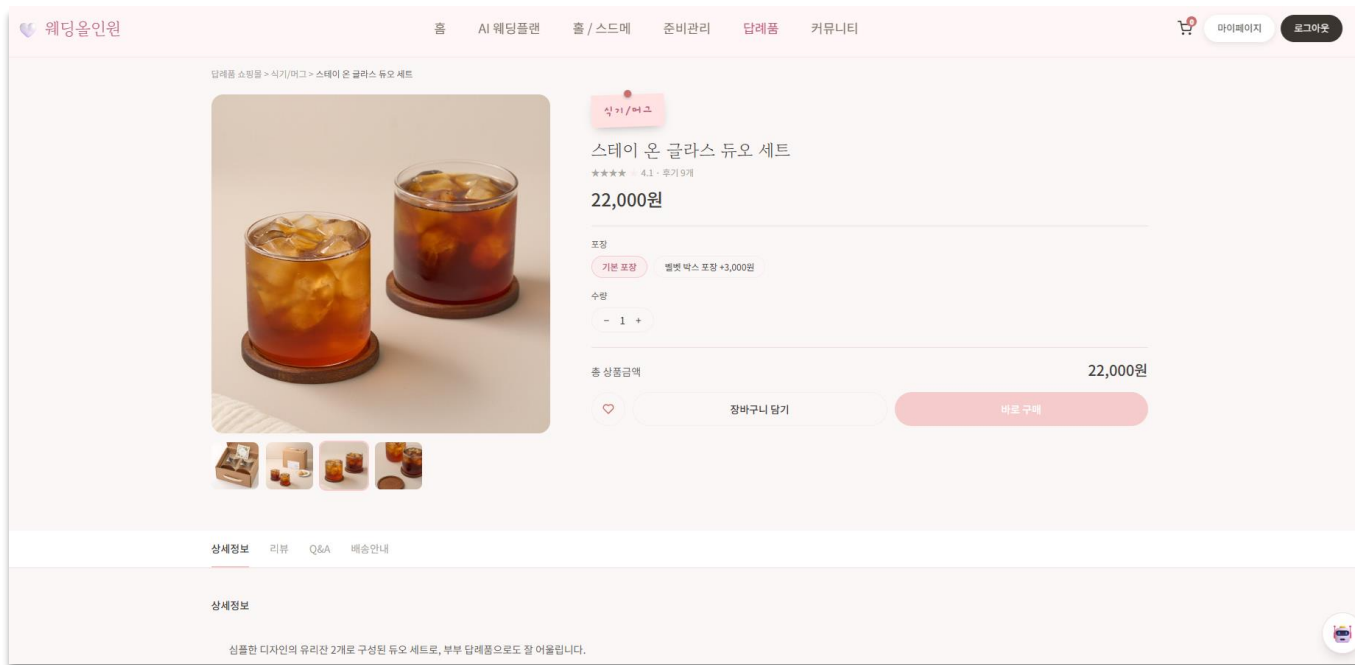
After



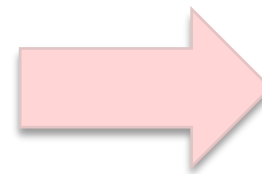
핵심

- 모든 페이지 코드를 한꺼번에 불러오지 않고, 각 페이지를 React.lazy로 감싸서 해당 페이지에 실제로 들어갈 때만 코드를 불러오게 만든다.
- 그 코드가 아직 불러오지 않은 짧은 순간에 화면에 무엇을 보여줄지는 Suspense 가 담당한다. (로딩 중 화면)
- 그 결과 사용자가 사이트에 처음 들어왔을 때는 지금 보고 있는 페이지 코드만 받으면 되고, 마이페이지 · 관리자 페이지처럼 아직 안 들어간 페이지 코드는 나중에 그 페이지로 이동할 때 받는다 → 첫 화면이 뜨는 속도가 빨라진다.

반응형 웹 구현



Before

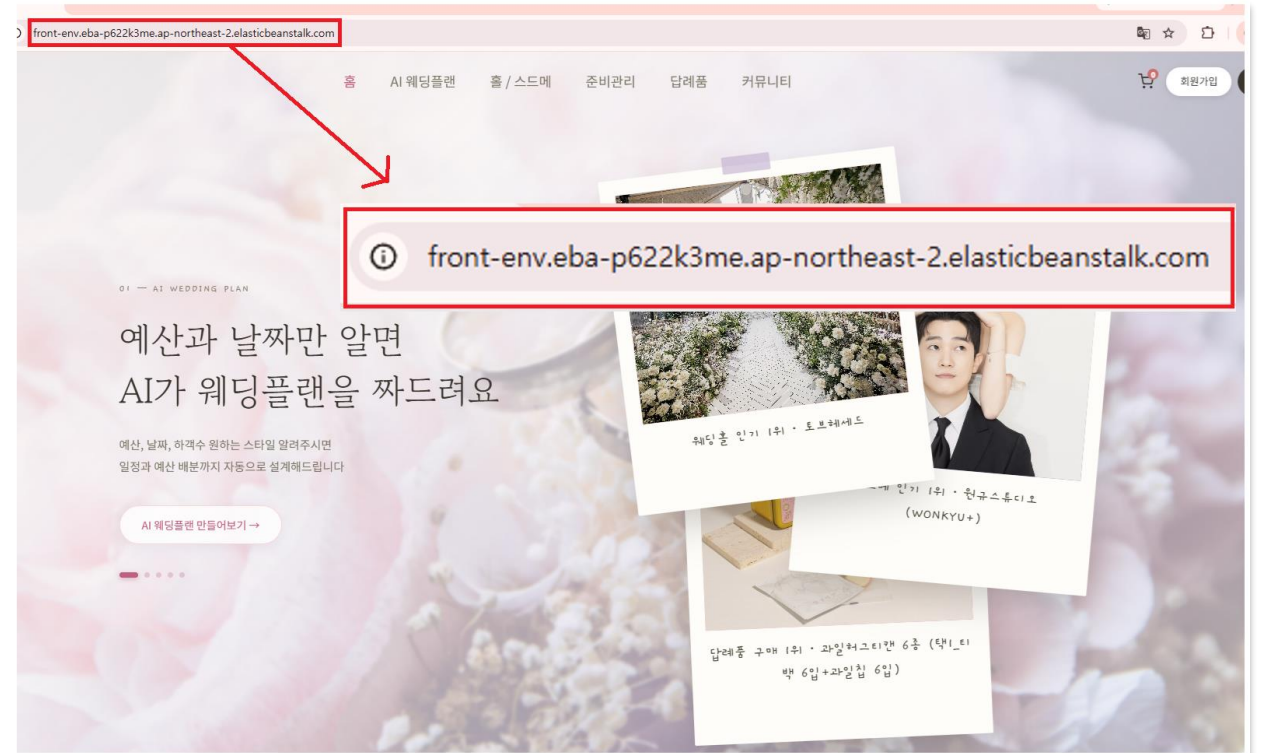
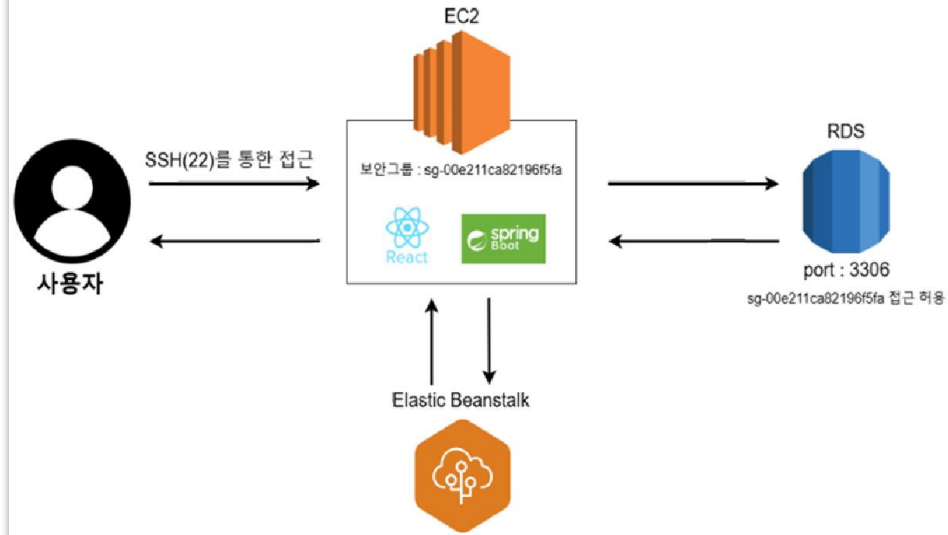


After

핵심

- PC, 태블릿, 모바일 등 다양한 디바이스 환경에 맞춰 레이아웃이 자동으로 변경되도록 구현하여 어떤 화면에서도 최적의 UI를 제공
- 화면크기에 따라 컴포넌트의 크기와 배치, 여백을 유동적으로 조정해 가독성과 사용성을 높이고 직관적인 사용자 경험을 제공
- 다양한 해상도와 브라우저 환경에서도 일관된 UI와 UX를 유지할 수 있도록 반응형 웹을 적용하여 접근성과 호환성을 향상

AWS 배포



핵심

- EC2와 Elastic Beanstalk을 활용하여 애플리케이션을 배포하고, 일관된 서버 환경에서 안정적으로 서비스를 운영
- RDS를 연동하여 애플리케이션과 데이터베이스를 분리하고, 데이터의 안정적인 저장 및 관리 환경을 구축
- IAM을 통해 사용자 및 권한을 관리하여 보안을 강화하고, AWS 리소스에 대한 접근 권한을 안전하게 제어

트러블슈팅 ①

! 문제

하드코딩된 서명 키로 관리자 토큰 위조 가능

🔍 원인

JWTUtil.java에 고정 문자열로 키 저장

✔ 해결

JWT_SECRET 환경변수 분리·랜덤 키 교체

@Log4j2 48개 사용 위치 DESKTOP-OT238HQWEZEN +1 +

public class JWTUtil {

 private static String key = "1234567890123456789012345678901234567890";
 서명 키가 소스코드에 문자열로 그대로 박혀있음 - 이 코드를 볼 수 있는 사람(또는 유출 시 누구나)
 이 값으로 유효한 토큰을 위조할 수 있음

ngApplication.java

application.properties ×
jwt.secret=\${JWT_SECRET:fDRTHWzPowDtw1UKifvqtXlyavUTBv5tMUckAIB0iVIXXEcx}
서명 키를 소스코드가 아닌 application.properties(환경 변수 JWT_SECRET)에서 주입받도록 분리.
스프링이 기동 시점에 setKey()를 통해 한 번만 값을 채워줌
- 코드에는 실제 키 값이 전혀 남지 않음

Before

After

Bearer Token

Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ

200 OK

JSON

1 {
2 "dtolist": [
3 {
4 "email": "user35@naver.com",
5 "nickname": "영통한곰돌이035",
6 "regDate": "2026-07-23T22:45:23",
7 "lastLoginAt": "2026-04-19T01:55:58",
8 "status": "ACTIVE",
9 "suspendReason": null,
10 "suspendUntil": null,
11 "emailVerified": true,
12 "admin": false
13 },
14],
15 "email": "user80@naver.com",
16 "nickname": "시원하버우"

로그인 절차 건너뛰고 관리자 토큰 생성 가능

401 Unauthorized

JSON

1 {
2 "error": "ERROR_ACCESS_TOKEN"
3 }
4

동일한 위조 토큰 재요청
-> 서명 불일치로 401

트러블슈팅 ②

! 문제

결제 승인 API 중복 호출 시 409 오류 노출

🔍 원인

PAID 확인 없이 토스 승인 API를 반복 호출

✓ 해결

PAID면 기존 결제 결과를 즉시 반환

```
public ReservationDTO confirmPayment(Long reservationId, String memberEmail,
    if ("PAID".equals(reservation.getPayStatus())) {
        return toDTO(reservation);
    } 이미 결제 완료 상태면 토스 재승인 호출 없이 기존 결과를 즉시 반환 - 중복 확정 요청을 여기서 차단
```

```
public class ReservationServiceImpl implements ReservationService {
    public List<ReservationDTO> confirmBulkPayment(String memberEmail, ReservationBulkPaymentConfirmRequestDTO requestDTO) {
        List<Reservation> targets = requestDTO.getReservationIds().stream().map(Long id -> reservationRepository.findById(id).orElseThrow()).toList();
        boolean alreadyConfirmed = targets.stream().allMatch(reservation r -> "PAID".equals(r.getPayStatus()) && requestDTO.getOrderNumber().equals(r.getOrderNumber()));
        if (alreadyConfirmed) {
            return targets.stream().map(reservation r -> modelMapper.map(r, ReservationDTO.class)).toList();
        } 묶음 결제도 같은 주문번호로 묶임 예약들이 이미 전부 결제완료 상태면, 토스 호출 없이 그대로 반환 - 단건 결제와 동일한 원칙을 묶음결제에도 적용
```

Before

POST http://localhost:8080/api/reservations/3391/payment/confirm

Body: {"paymentKey": "tviva202607271047060Rcx9", "orderNumber": "RSV-3391-1785116826759", "amount": 44000}

Test Results: 409 Conflict

Body: {"msg": "결제 승인에 실패했습니다: {\code: \"ALREADY_PROCESSED_PAYMENT\", \"message: \"이미 처리된 결제 입니다.\"}

After

POST http://localhost:8080/api/reservations/3391/payment/confirm

Body: {"paymentKey": "tviva202607271047060Rcx9", "orderNumber": "RSV-3391-1785116826759", "amount": 44000}

Test Results: 200 OK

Body: {"reservationId": 3391, "memberEmail": "user26@naver.com", "cmno": 4, "weddingDate": "2026-08-13", "status": "확정", "memo": "", "optionName": "베이직 슈트", "amount": 44000, "orderNumber": "RSV-3391-1785116826759", "paymentKey": "tviva202607271047060Rcx9", "payStatus": "PAID", "paidAt": "2026-07-27 10:47:28", "paymentDeadline": "2026-07-30"}

동일 상황에서도 에러 alert 없이 결제 완료 화면 정상 노출 -> 사용자 혼란 - 불필요한 CS문의 예방

트러블슈팅 ③

! 문제

제외(X)한 업체를 '다시 찾기'로 되돌리면, 원래 업체가 아닌 매번 다른 업체·추천 사유가 무작위로 표시됨

🔍 원인

제외 처리 시 선택 업체·추천 사유를 함께 초기화(null)해, 복원할 정보가 없어 매번 새로 검색·재생성

✓ 해결

제외 시 상태만 EXCLUDED로 바꾸고 선택 정보는 보존 → '다시 찾기'는 새로 검색하지 않고 원래 업체·사유를 그대로 복원

```
case "EXCLUDE" -> {  
    slot.changeStatus(SlotStatus.EXCLUDED);  
    slot.changeSelectedCmno(null);  
}  
default -> slot.changeStatus(SlotStatus.PENDING);  
}
```

제거



BEFORE

제외 후 다시 찾기
매번 다른 업체 표시

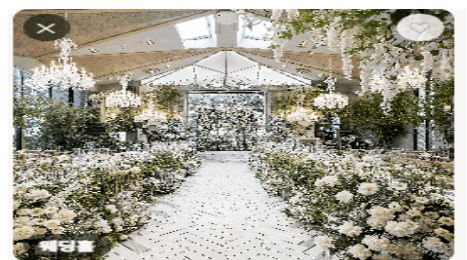


웨딩홀 · 토브헤세드
플로렌스홀
18,000,000원
토브헤세드는 최대 300명을 수용할 수 있는 넓은 단독 홀로, 나만의 프라이빗 맞춤웨딩을 통해 하객들에게 특별한 경험을 제공합니다.
업체 확정됨 · 해제하기



AFTER

제외 후 다시 찾기
원래 업체 그대로 복원



웨딩홀 · 토브헤세드
플로렌스홀
18,000,000원
토브헤세드는 최대 300명을 수용할 수 있는 넓은 단독 홀로, 나만의 프라이빗 맞춤웨딩을 통해 하객들에게 특별한 경험을 제공합니다.
이 업체로 확정하기

회고

주요 성과

- **AI 4종 통합** — 웨딩플랜·챗봇·드레스·견적서를 하나의 플랫폼에서 유기적으로 연결
- **실시간 커뮤니케이션 구축** — WebSocket(STOMP) 기반 업체-회원 실시간 문의 채팅 완성
- **성능 최적화 3종 적용** — N+1 방지, 서버 사이드 페이징, 코드 스플리팅으로 체감 속도 개선

배운 점

- **보안은 나중에 아니라 처음부터** — JWT 시크릿 하드코딩을 겪으며 설정값 외 부화 습관의 중요성 체감
- **"동작한다" ≠ "안전하다"** — 결제 승인 API는 여러 번 호출돼도 같은 결과가 나오게 처음부터 설계해야 함을 체득
- **초기 DB 설계는 완성본이 아니었다** — 개발 중 필드는 물론 테이블까지 새로 추가되며 유연한 스키마 설계의 중요성 체감

아쉬운 점

- **코드 컨벤션 미통일** — 팀원별 주석 스타일 차이로 리뷰 시 맥락 파악에 시간 소요
- **테스트 자동화 미흡** — 트러블슈팅 재현을 수동 캡처에 의존, 회귀 테스트 부재

향후 계획

- **로그인/회원가입** — 카카오 하나뿐인 소셜 로그인에 네이버·구글 추가, 휴대폰 본인인증(SMS) 연동으로 이메일 인증만으론 못 막는 허위가입까지 방지
- **AI견적서** — 2개 견적서 비교만 가능한 걸 3개 이상 동시 비교로 확장 항목별 시세 데이터와 비교해서 “이 항목이 평균보다 비쌉니다” 같은 인사이트 제공
- **AI운영관제** — 이상 감지 시 텔레그램/슬랙으로 실시간 알림 발송, 주간 브리핑에 전주 대비 추이그래프 추가

Thank you